

```

import crypto from 'node:crypto';
import { q, one, j } from '../db.js';
import { defineCapability } from './policy.js';
import { emit } from './events.js';

const hash = (v) => crypto.createHash('sha256').update(JSON.stringify(v)).digest('hex');

// Authority ranking. Lower number wins. The owner is always rank 1.
export const AUTHORITY = {
  owner: 1, staff: 2, pos: 3, booking: 4, connector: 5,
  website: 6, directory: 7, scrape: 8, inference: 9,
};

// --- capabilities -----

defineCapability({
  key: 'business.set_fact',
  packageKey: 'kernel',
  summary: 'Set a canonical fact about the business.',
  route: 'internal',
  handler: async ({ ctx, input }) => {
    const { key, value, locationId = null } = input;
    await q(
      `update business_fact set effective_to = now()
       where business_id = $1 and key = $2 and effective_to is null`,
      [ctx.businessId, key]
    );
    const row = await one(
      `insert into business_fact (business_id, location_id, key, value, set_by)
       values ($1,$2,$3,$4,$5) returning *`,
      [ctx.businessId, locationId, key, JSON.stringify(value), ctx.person?.id ?? null]
    );
    await recordObservation({
      businessId: ctx.businessId, source: 'owner', authorityRank: AUTHORITY.owner,
      subject: 'business', key, value,
    });
    await emit({ businessId: ctx.businessId, topic: 'canonical.fact_changed', payload: value });
    return row;
  },
  verify: async ({ ctx, input }) => {
    const row = await one(
      `select value from business_fact
       where business_id = $1 and key = $2 and effective_to is null`,
      [ctx.businessId, input.key]
    );
  }
});

```

```

    );
    const stored = j(row?.value);
    const ok = row && hash(stored) === hash(input.value);
    return {
      state: ok ? 'verified' : 'failed',
      evidence: [{ kind: 'read_back', key: input.key, value: stored ?? null }],
    };
  },
});

defineCapability({
  key: 'business.set_hours',
  packageKey: 'kernel',
  summary: 'Replace the weekly hours for a location.',
  route: 'internal',
  handler: async ({ ctx, input }) => {
    const { locationId = null, days } = input; // days: [{weekday, opens, closes,
    await q(`delete from regular_hours where business_id = $1 and (location_id =
      [ctx.businessId, locationId]);
    const rows = [];
    for (const d of days) {
      rows.push(await one(
        `insert into regular_hours (business_id, location_id, weekday, opens, clo
          values ($1,$2,$3,$4,$5,$6) returning *`,
        [ctx.businessId, locationId, d.weekday, d.opens ?? null, d.closes ?? null
      ));
    }
    await recordObservation({
      businessId: ctx.businessId, source: 'owner', authorityRank: AUTHORITY.owner
      subject: 'hours', key: 'regular_hours', value: days,
    });
    return rows;
  },
  verify: async ({ ctx, input }) => {
    const rows = await q(
      `select weekday, closed from regular_hours where business_id = $1 order by
    );
    return {
      state: rows.length === input.days.length ? 'verified' : 'partial',
      evidence: [{ kind: 'row_count', expected: input.days.length, actual: rows.l
    };
  },
});

defineCapability({
  key: 'business.set_temporary_closure',
  packageKey: 'kernel',

```

```

summary: 'Close or alter hours for a bounded window without touching regular ho
route: 'internal',
sensitivity: 'high',
handler: async ({ ctx, input }) => {
  const row = await one(
    `insert into temporary_hours (business_id, location_id, starts_at, ends_at,
    values ($1,$2,$3,$4,$5,$6,$7,$8) returning *`,
    [ctx.businessId, input.locationId ?? null, input.startsAt, input.endsAt,
    input.opens ?? null, input.closes ?? null, input.closed ?? true, input.rea
  );
  await markAllChannelsStale({ businessId: ctx.businessId, key: 'hours' });
  return row;
},
verify: async ({ ctx, result }) => {
  const row = await one(`select * from temporary_hours where id = $1`, [result.
  return { state: row ? 'verified' : 'failed', evidence: [{ kind: 'row', id: re
},
]);

// --- provenance -----

export async function recordObservation({ businessId, connectionId = null, source
  return one(
    `insert into observation (business_id, connection_id, source, authority_rank,
    values ($1,$2,$3,$4,$5,$6,$7) returning *`,
    [businessId, connectionId, source, authorityRank, subject, key, JSON.stringify
  );
}

// What a channel currently believes is never a column on the business.
// It is a separate observation, compared against canonical to produce drift.
export async function drift({ businessId, key }) {
  const canonical = await one(
    `select value from business_fact where business_id = $1 and key = $2 and effe
    [businessId, key]
  );
  const canonicalValue = j(canonical?.value);
  const canonicalHash = canonical ? hash(canonicalValue) : null;

  const channels = await q(
    `select id, provider_key from connection where business_id = $1 and status =
    [businessId]
  );

  const out = [];
  for (const c of channels) {
    const latest = await one(

```

```

        `select value from observation
          where connection_id = $1 and key = $2
          order by observed_at desc limit 1`,
        [c.id, key]
      );
      const v = j(latest?.value);
      out.push({
        provider: c.provider_key,
        value: v ?? null,
        inSync: v !== undefined && v !== null && hash(v) === canonicalHash,
      });
    }

    return { key, canonical: canonicalValue ?? null, channels: out };
  }

  async function markAllChannelsStale({ businessId, key }) {
    await q(
      `update channel_sync_state set in_sync = false
        where business_id = $1 and key = $2`, [businessId, key]
    );
  }
}

```